

ASSIGNMENT : 3

TITLE : Develop a Java application demonstrating method overloading and the use of static variables, methods in class design.

Problem Statement : Develop a Java application demonstrating method overloading and the use of static variables, methods in class design.

Learning Objectives : To implement method overloading and use static variables and methods in Java programs.

Theory : Method overloading allows multiple methods to have the same name with different parameter lists, enabling compile-time polymorphism. The static keyword is used for variables and methods that belong to the class instead of individual objects. Static members share common data among all objects and reduce memory usage. These concepts improve code flexibility, readability, and efficient resource management.

Exercise :

1. Develop a **Calculator program** using overloaded methods for addition of integers and decimals. Use a static variable to count calculations.

```
1  class Calculator {
2      static int calculationCount = 0;
3
4      // Addition of integers
5      int add(int a, int b) {
6          calculationCount++;
7          return a + b;
8      }
9
10     // Addition of decimal numbers
11     double add(double a, double b) {
12         calculationCount++;
13         return a + b;
14     }
15
16     static void displayCount() {
17         System.out.println("Total Calculations: " + calculationCount);
18     }
19 }
20
21 public class Main {
22     public static void main(String[] args) {
23
24         Calculator calc = new Calculator();
25
26         System.out.println("Integer Addition: " + calc.add(10, 20));
27         System.out.println("Decimal Addition: " + calc.add(12.5, 7.8));
28
29         Calculator.displayCount();
30     }
31 }
```

Output

Generated files

```
Integer Addition: 30
Decimal Addition: 20.3
Total Calculations: 2
|
```

2. Develop a **Restaurant Billing Application** where overloaded methods calculate bills for dine-in, takeaway, and delivery orders, while static variables track total orders.

```
class Restaurant {
    static int totalOrders = 0;

    double calculateBill(double amount) {
        totalOrders++;
        return amount;
    }

    double calculateBill(double amount, double packingCharge) {
        totalOrders++;
        return amount + packingCharge;
    }

    double calculateBill(double amount, double packingCharge, double deliveryCharge) {
        totalOrders++;
        return amount + packingCharge + deliveryCharge;
    }

    static void displayOrders() {
        System.out.println("Total Orders: " + totalOrders);
    }
}

public class Main {
    public static void main(String[] args) {

        Restaurant order = new Restaurant();

        System.out.println("Dine-in Bill: ₹" + order.calculateBill(500));
        System.out.println("Takeaway Bill: ₹" + order.calculateBill(500, 30));
        System.out.println("Delivery Bill: ₹" + order.calculateBill(500, 30, 50));

        Restaurant.displayOrders();
    }
}
```

Output

Generated files

```
Dine-in Bill: ₹500.0  
Takeaway Bill: ₹530.0  
Delivery Bill: ₹580.0  
Total Orders: 3
```

Outcome

- Understood and implemented **method overloading** in Java.
- Learned the use of **static variables** and **static methods** to share data among all objects.
- Developed Java programs demonstrating compile-time polymorphism and class-level members.

Conclusion

The assignment demonstrated how **method overloading** improves code flexibility by allowing methods with the same name to perform similar operations using different parameters. It also showed that **static variables and methods** belong to the class, enabling shared data management and efficient resource utilization in Java applications.